



NANDHA ENGINEERING COLLEGE

(AUTONOMOUS), ERODE – 638 052



AICTE IDEA LAB

WEB DEVELOPMENT REFERENCE MANUAL

Foundation Laboratory (WEB-X)

26GEP01

```
<!DOCTYPE html>
<h1 class="hello">Hello, WEB-X</h1>
.hello { color: #e8232f; }
$ git add . && git commit && git push
```

HTML5

CSS3

JavaScript

Git & Cloud



LEARN



BUILD



INNOVATE



TRANSFORM



NANDHA ENGINEERING COLLEGE

(AUTONOMOUS), ERODE – 638 052



• INSTITUTE VISION AND MISSION •



VISION

To be an Institute of excellence providing quality Engineering, Technology and Management education to meet the ever changing needs of the society.



MISSION

- To provide quality education to produce ethical and competent professionals with social responsibility.
- To excel in the thrust areas of Engineering, Technology and Entrepreneurship by solving real-world problems.
- To create a learner centric environment and improve continually to meet the changing global needs.

• NEC AICTE IDEA LAB VISION AND MISSION •



VISION

To be a leading centre for innovation and multidisciplinary product development through Design Thinking for skill enhancement.



MISSION

- Upskilling the young minds with advanced skills in prototyping.
- Creating strong partnership with industries, investors and startups.
- Active participation in global innovation challenges and entrepreneurship forums.



PROGRAM
EDUCATIONAL
OBJECTIVES
(PEOs)

The graduates of Electrical and Electronics Engineering will be

PEO1: Core Competency: A successful professional with domain knowledge in Electrical and Electronics Engineering using emerging techniques.

PEO2: Research, Innovation and Entrepreneurship: Able to demonstrate multi-disciplinary skills through innovation and research to meet the societal needs.

PEO3: Ethics, Human values and Life-long learning: Able to demonstrate ethical practices and managerial skills through continual learning.



PROGRAM
SPECIFIC
OUTCOMES
(PSOs)

The students of Electrical and Electronics Engineering will be able to

PSO1: Analyze, design and validate processes, products by applying knowledge and skills in Power system, Electrical Machines and Power Electronics.

PSO2: Design and analyze the processes of smart grid and renewable energy systems using appropriate tools and techniques.



PROGRAM OUTCOMES (POs)



PROGRAM OUTCOMES (POs)

PO1: Engineering Knowledge

Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop solutions to complex engineering problems.

PO2: Problem Analysis

Identify, formulate, review research literature and analyze complex engineering problems, reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions

Design creative solutions for complex engineering problems and design/develop systems, components or processes to meet identified needs with consideration for public health and safety, whole-life cost, net-zero carbon, culture, society and the environment, as required. (WK5)

PO4: Conduct Investigations of Complex Problems

Conduct investigations of complex engineering problems using research-based knowledge, including the design of experiments, modeling, analysis and interpretation of data, to provide valid conclusions. (WK8)

PO5: Engineering Tool Usage

Create, select and apply appropriate techniques, resources and modern engineering and IT tools, including prediction and modeling, recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and the World

Analyze and evaluate societal and environmental aspects while solving complex engineering problems for their impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5 and WK7)

PO7: Ethics

Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national and international laws. (WK9)

PO8: Individual and Collaborative Team Work

Function effectively as an individual and as a member or leader in diverse and multidisciplinary teams.

PO9: Communication

Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, and make effective presentations considering cultural, language and learning differences.

PO10: Project Management and Finance

Apply knowledge and understanding of engineering management principles and economic decision-making, and apply these to one's own work as a member and leader in a team, and to manage projects in multidisciplinary environments.

PO11: Life-Long Learning

Recognize the need for, and have the preparation and ability for:

1. Independent and life-long learning.
2. Adaptability to new and emerging technologies.
3. Critical thinking in the broadest context of technological change. (WK8)

26GEP01-Foundation Laboratory (Electrical, IoT and WebX)							
Course Code:					CIA		100
Course Type: (Theory/Practical/Integrated)		Practical			ESE		-
L:T:P:C		0	0	60	2	Total Marks	100
Total Hours of Pedagogy		60			Exam Hours		-
Pre-requisites		NIL					
Course Objectives							
To provide hands-on experience in: - Developing fundamental practical competencies in electrical wiring, protection, motor control, measurement, and IoT-based applications using standard engineering tools and safe working practices. - Designing, fabricating, assembling, and testing single-layer PCB circuits, and developing IoT-enabled embedded applications using modern development platforms. - Developing responsive and interactive web application prototypes using HTML, CSS, and JavaScript for simple monitoring and control applications.							
Course Outcomes The students will be able to						Cognitive Level	Weightage of COs in CIA
CO 1	Assemble, test, and troubleshoot electrical wiring, protection, motor control, measurement, and IoT-based systems using appropriate tools and safe engineering practices.					Ap	33
CO 2	Develop and assemble a single-layer PCB circuits and implement an IoT system with sensor interfacing.					An	33
CO 3	Develop responsive and interactive web application prototypes using HTML, CSS, and JavaScript for simple monitoring and control applications.					Ap	34

Mapping of COs with POs / PSOs													
COs	POs											PSOs	
	1	2	3	4	5	6	7	8	9	10	11	1	2
CO 1	3		3		3							3	
CO 2	3	3			3								3
CO 3	3		3		3				2			3	
CO (W.A)	3	3	3		3				2			3	3

List of Experiments:

(i) Electrical

1. Design and development of domestic electrical wiring and protection system.
2. Design and implementation of electrical measurement and motor control system.
3. Development of smart electrical monitoring and load control system using ESP32.

(ii) IoT & PCB Design

1. Designing a single layer PCB layout.
2. Fabrication and assembling a single layer PCB.
3. Integration of IoT devices with cloud platform.

(iii) WebX

1. Design basic web pages using HTML and CSS.
2. Create Interactive web pages using JavaScript.
3. Develop a web application prototype.

Total (P : 60)

Reference Books:

1. IS 732:2019 – Code of Practice for Electrical Wiring Installations, Bureau of Indian Standards, 2019.
2. S. L. Uppal and G. C. Garg, Electrical Wiring, Estimating and Costing, Khanna Publishers, New Delhi, 2021.
3. B. L. Theraja and A. K. Theraja, Electrical Technology – Volume I, S. Chand Publishing, 2022.
4. R. S. Khandpur, Printed Circuit Boards: Design, Fabrication, Assembly and Testing, McGraw Hill Education India Pvt. Ltd., 2019.
5. Arshdeep Bahga and Vijay Madisetti, Internet of Things – A Hands-on Approach, Universities Press, 2015.
6. Simon Monk, Programming with ESP32, McGraw Hill, 2022.
7. Jon Duckett, HTML and CSS: Design and Build Websites, Wiley Publications, 2014.
8. Jon Duckett, JavaScript and JQuery: Interactive Front-End Web Development, Wiley Publications, 2015.
9. Marco Schwartz, Internet of Things with ESP8266, Packt Publishing, 2016.

HTML

CSS

JS

Web Development in 3 Days

HTML + CSS + JavaScript for Absolute Beginners

First Year Student Handbook | What? Why? How?

No prior knowledge needed. Every topic is explained from zero.

Table of Contents

Find any topic quickly using the page number in the last column.

HTML, CSS and JavaScript

S.No	Title	Page No.
1	Welcome!	3
2	3-Day Plan	3
3	Human Body Analogy (remember this!)	3
4	Before We Start: Setup (10 minutes)	3
Day 1: HTML - Structure of a web page		4
5	What is HTML?	4
6	Your First Web Page	4
7	Headings and Paragraphs	5
8	Text Formatting	5
9	Links (Anchor Tag)	5
10	Images	6
11	Lists	6
12	Tables	6
13	Forms and Inputs	7
14	Semantic Tags and div / span	7
15	Comments, IDs and Classes	8
16	Day 1 Project: My Profile Page	8
Day 2: CSS - Style and layout of a web page		10
17	What is CSS?	10
18	Selectors	10
19	Colors and Backgrounds	11
20	Text and Fonts	11
21	Units: px, %, em, rem, vw	11
22	The Box Model (Most Important!)	12
23	Display: block, inline, inline-block, none	12
24	Flexbox: Easy Layouts	13
25	Position	13
26	Hover Effects and Transitions	14
27	Responsive Design (Media Queries)	14
28	Day 2 Project: Style Your Profile Page	14
Day 3: JavaScript - Behaviour and interaction		16
29	What is JavaScript?	16
30	Variables	16
31	Data Types	16
32	Operators	17
33	Conditions (if / else)	17
34	Loops	17
35	Functions	18
36	Arrays	18
37	Objects	18
38	The DOM: Changing the Web Page	19
39	Events	19
40	Day 3 Mini Projects	19
41	Common Beginner Mistakes	22
42	How to debug (find mistakes)	22
43	One-Page Cheat Sheet	22
44	Glossary of Simple Words	23
45	What to Learn Next	23

Git, GitHub and Netlify

S.No	Title	Page No.
46	Welcome!	25
47	The Big Picture	25
48	Real-Life Analogy	25
49	Before You Start: Setup (15 minutes)	25
Part 1: Git - Save versions of your code on your computer		26
50	What is Git?	26
51	Terminal Basics (needed for Git)	26
52	One-Time Setup: Tell Git Who You Are	26
53	The Three Areas of Git (Very Important)	27
54	Start Tracking: git init	27
55	Check the Situation: git status	27
56	Stage Files: git add	27
57	Save a Snapshot: git commit	28
58	See History: git log and git diff	28
59	Ignore Files: .gitignore	28
60	Undo Mistakes Safely	28
61	Branches	29
62	The Everyday Git Cycle	29
Part 2: GitHub - Back up and share your code online		30
63	What is GitHub?	30
64	Create a Repository on GitHub	30
65	Logging In from Your Computer	30
66	Connect and Push Your Project	31
67	The Daily Workflow	31
68	Pull and Clone	31
69	README: The Front Page of Your Project	32
70	Collaboration Basics	32
71	Common Errors and Fixes	33
Part 3: Netlify - Host your website on the internet		34
72	What are Hosting and Netlify?	34
73	Pre-Deployment Checklist	34
74	Method A: Drag and Drop (Fastest, 2 Minutes)	35
75	Method B: Deploy from GitHub (Recommended)	35
76	Update Your Live Site	35
77	Change the Site Name	36
78	Custom Domain (Optional)	36
79	Troubleshooting Deployments	36
80	Your Final Project Checklist	37
81	Git and GitHub Cheat Sheet	38
82	Glossary of Simple Words	38
83	What to Learn Next	39

Welcome!

This handbook takes you from **zero** to building a real, working web page in three days. You do not need to know anything about coding. Every topic follows the same pattern:

Question	What it answers
WHAT?	What is this thing? (simple definition)
WHY?	Why do we need it? (the problem it solves)
HOW?	How do we use it? (syntax + example code)

3-Day Plan

Day	Language	Big idea	You will build
Day 1	HTML	Structure: the skeleton of a page	A personal profile page
Day 2	CSS	Style: colors, layout, design	A styled, responsive profile page
Day 3	JavaScript	Behaviour: make the page react	Counter, color changer, to-do list

Human Body Analogy (remember this!)

Part of body	Web language	Job
Skeleton / bones	HTML	Holds the content: headings, text, images, buttons
Clothes / looks	CSS	Makes it beautiful: colors, fonts, spacing, layout
Muscles / brain	JavaScript	Makes it move and think: clicks, calculations, updates

Before We Start: Setup (10 minutes)

- Install a **browser**: Google Chrome (best for learning, has great developer tools).
- Install a **code editor**: Visual Studio Code (free) from code.visualstudio.com.
- Create a folder on your Desktop called **web-course**. All your files go here.
- In VS Code: File > Open Folder > choose **web-course**.
- Optional but useful: install the **Live Server** extension. It refreshes the browser every time you save.

How web pages actually work

When you type a website address in the browser, your computer (the **client**) asks another computer (the **server**) for files. The server sends back HTML, CSS and JS files, and the **browser** reads them and draws the page on your screen.

```
You (Browser) --- "give me google.com" ---> Server
You (Browser) <--- HTML + CSS + JS files --- Server
Browser reads the files and shows the page.
```

TIP: Save your file, then press F5 (or Ctrl+R) in the browser to see the changes. Do this after every small change. Beginners who test often make fewer mistakes.

HTML

Structure of a web page

1. What is HTML?

WHAT? HTML stands for **HyperText Markup Language**. It is not a programming language; it is a markup language that labels content (this is a heading, this is a paragraph, this is an image).

WHY? Without HTML a browser only sees plain text. HTML tells the browser what each piece of content is.

HOW? You wrap content inside **tags**. Most tags come in pairs: an opening tag and a closing tag with a slash.

```
<tagname>  content goes here  </tagname>
  ^                               ^
opening tag                   closing tag
```

```
<p>Hello, I am a paragraph.</p>
```

- **Element** = opening tag + content + closing tag.
- **Attribute** = extra information inside the opening tag, like `href="..."`.
- Some tags have no closing tag (called **void tags**): `<br>`, `<hr>`, `<img>`, `<input>`.

2. Your First Web Page

WHAT? Every HTML page has the same basic skeleton (structure).

WHY? The browser needs this skeleton to understand your file properly.

HOW? In VS Code create a file named `index.html`, paste the code below, save, then double-click the file to open it in the browser.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello World!</h1>
  <p>This is my first web page.</p>
</body>
</html>
```

Part	Meaning
<code><!DOCTYPE html></code>	Tells the browser: this is a modern HTML5 document.
<code><html></code>	Root element. Everything lives inside it. <code>lang="en"</code> = English.
<code><head></code>	Hidden information: title, character set, CSS links. Not shown on the page.

Part	Meaning
<code><meta charset></code>	Lets the browser show all letters and symbols correctly.
<code><meta viewport></code>	Makes the page fit properly on mobile screens.
<code><title></code>	Text shown on the browser tab.
<code><body></code>	Everything the visitor SEES goes here.

3. Headings and Paragraphs

WHAT? Headings (**h1** to **h6**) are titles. **p** is a paragraph of text.

WHY? They give the page an outline. Search engines and screen readers depend on headings to understand your content.

HOW? Use **h1** once per page for the main title, **h2** for sections, **h3** for sub-sections, and so on.

```
<h1>Main Title</h1>
<h2>Section Title</h2>
<h3>Sub-section Title</h3>
<p>This is a paragraph. Browsers add space above and below it.</p>
<br>          <!-- line break -->
<hr>          <!-- horizontal line -->
```

TIP: Never choose a heading just because it looks big or small. Choose it for meaning. You can change size later with CSS.

4. Text Formatting

WHAT? Small tags that change how a piece of text is understood.

WHY? To emphasise words, mark important text, or show quotes.

HOW? Wrap the words you want to change.

Tag	Result	Use
<code> / </code>	bold	Important text (strong has meaning, b is only look)
<code><i> / </code>	italic	Emphasised text
<code><u></code>	underline	Underlined text
<code><mark></code>	highlighted	Highlight a word
<code><small></code>	smaller	Fine print
<code> / <ins></code>	struck / inserted	Removed / added text
<code><sub> / <sup></code>	H2O / x2	Sub and superscript

```
<p>I am <strong>learning</strong> <em>web development</em>.</p>
<p>Water is H<sub>2</sub>O and 5<sup>2</sup> = 25.</p>
```

5. Links (Anchor Tag)

WHAT? The `<a>` tag creates a clickable link. It is the reason it's called *hyper*text.

WHY? Links connect pages together. Without them there would be no 'web'.

HOW? Use the `href` attribute for the destination address. `target="_blank"` opens the link in a new tab.

```
<a href="https://www.google.com">Go to Google</a>
<a href="https://www.google.com" target="_blank">Open in new tab</a>
<a href="about.html">Go to my About page</a>      <!-- another file -->
<a href="#contact">Jump to the Contact section</a>  <!-- same page -->
<a href="mailto:me@example.com">Email me</a>
```

6. Images

WHAT? The `<img>` tag shows a picture.

WHY? Pictures make pages attractive and easier to understand.

HOW? Put the picture file in your `web-course` folder, then point to it with `src`.

```


```

- `src` = source (file name or web address of the image).
- `alt` = alternative text shown if the image fails, and read aloud for blind users. Always write it.
- `width` and `height` set the size in pixels.

7. Lists

WHAT? Lists group related items.

WHY? Menus, steps, shopping items and features are all lists.

HOW? Choose ordered (numbers) or unordered (bullets). Each item is an `<li>`.

```
<ul>                                <ol>
  <li>HTML</li>                      <li>Open editor</li>
  <li>CSS</li>                       <li>Write code</li>
  <li>JavaScript</li>               <li>Test it</li>
</ul>                                </ol>
(bullets)                          (1. 2. 3.)
```

8. Tables

WHAT? A table shows data in rows and columns.

WHY? Timetables, marks, price lists.

HOW? `<table>` holds everything, `<tr>` is a row, `<th>` a heading cell, `<td>` a normal cell.

```
<table border="1">
  <tr>
    <th>Name</th>
    <th>Marks</th>
  </tr>
  <tr>
    <td>Arun</td>
    <td>85</td>
  </tr>
  <tr>
    <td>Meena</td>
    <td>92</td>
  </tr>
</table>
```

TIP: `colspan="2"` makes a cell stretch across 2 columns. `rowspan="2"` stretches down 2 rows.

9. Forms and Inputs

WHAT? A form collects information from the user.

WHY? Login pages, sign-ups, feedback, search boxes.

HOW? Put `<input>` fields inside `<form>`. Pair every field with a `<label>`.

```
<form>
  <label for="name">Name:</label>
  <input type="text" id="name" placeholder="Enter your name">

  <label for="pw">Password:</label>
  <input type="password" id="pw">

  <label for="mail">Email:</label>
  <input type="email" id="mail" required>

  <p>Gender:</p>
  <input type="radio" name="g" value="m"> Male
  <input type="radio" name="g" value="f"> Female

  <p>Hobbies:</p>
  <input type="checkbox"> Music
  <input type="checkbox"> Sports

  <label for="c">Course:</label>
  <select id="c">
    <option>BCA</option>
    <option>B.Sc CS</option>
  </select>

  <textarea rows="3" cols="30">Your message</textarea>
  <button type="submit">Submit</button>
</form>
```

Input type	What it gives
text	Single-line text box
password	Hides typed letters
email	Checks for a valid email
number	Numbers only
radio	Choose ONE from a group (same name)
checkbox	Choose many
date	Date picker
submit	Send button

10. Semantic Tags and div / span

WHAT? Semantic tags are tags whose names describe their meaning. `div` and `span` are neutral containers.

WHY? Clear names make code easier to read and help search engines and screen readers. Containers help us group things so CSS can style them later.

HOW? Use these tags to divide the page into logical parts.

```
<header>  Logo and menu at the top      </header>
<nav>     Navigation links              </nav>
<main>
```

```

    <section> A section of content                </section>
    <article> A complete post / story             </article>
    <aside>   Side content, ads                   </aside>
</main>
<footer>   Copyright, contact info              </footer>

<div> block container: starts on a new line, takes full width </div>
<span> inline container: sits inside a line of text          </span>

```

11. Comments, IDs and Classes

WHAT? Comments are notes for humans. **id** and **class** are name tags you give to elements.

WHY? Comments explain your code. Names let CSS and JavaScript find the exact element they want to change.

HOW? Write them as attributes.

```

<!-- This is a comment. The browser ignores it. -->
<h1 id="title">Unique heading</h1>           <!-- id: used ONCE per page -->
<p class="note">First note</p>               <!-- class: reused many times -->
<p class="note">Second note</p>

```

Day 1 Project: My Profile Page

Create **profile.html**. Use every tag below at least once. This page is reused on Day 2 and Day 3, so keep it safe.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Profile</title>
</head>
<body>
  <header>
    <h1>Arun Kumar</h1>
    <p>First Year Student</p>
  </header>

  <nav>
    <a href="#about">About</a> |
    <a href="#skills">Skills</a> |
    <a href="#contact">Contact</a>
  </nav>

  <main>
    <section id="about">
      <h2>About Me</h2>
      
      <p>I am learning <strong>web development</strong>.</p>
    </section>

    <section id="skills">
      <h2>My Skills</h2>
      <ul>
        <li>HTML</li>
        <li>CSS</li>
        <li>JavaScript</li>
      </ul>
    </section>

    <section id="contact">
      <h2>Contact</h2>
      <form>
        <label>Name: <input type="text"></label>
        <label>Message: <textarea></textarea></label>
      </form>
    </section>
  </main>
</body>
</html>

```

```
        <button>Send</button>
      </form>
    </section>
  </main>

  <footer><p>Made by Arun</p></footer>
</body>
</html>
```

Day 1 Practice Questions

- What does HTML stand for? Is it a programming language?
- Write the tag for the biggest heading.
- Difference between `<ul>` and `<ol>`?
- Why should every `<img>` have an `alt`?
- Difference between `<div>` and `<span>`?
- Create a table of 3 friends with name and age.

1. What is CSS?

WHAT? CSS stands for **Cascading Style Sheets**. It is the language that describes how HTML elements look.

WHY? Plain HTML looks like a boring document. CSS adds color, fonts, spacing and layout, and one CSS file can style hundreds of pages.

HOW? Write a **selector** (which element?) followed by curly braces holding **property: value;** pairs (what to change?).

```
selector {
  property: value;
  property: value;
}

h1 {
  color: red;           /* text color */
  font-size: 40px;      /* text size */
}
```

Three ways to add CSS

Method	Example	When to use
Inline	<code><p style="color:red;">Hi</p></code>	Quick test only. Avoid in real projects.
Internal	<code><style> p { color:red; } </style></code> inside <code><head></code>	Small single page.
External	<code><link rel="stylesheet" href="style.css"></code>	Best. Separate file, reused by every page.

Steps: create `style.css` in the same folder, then add this line inside `<head>` of `profile.html`:

```
<link rel="stylesheet" href="style.css">
```

2. Selectors

WHAT? A selector chooses which elements the style applies to.

WHY? Without selectors you could not style headings and paragraphs differently.

HOW? Use the pattern that fits your need:

Selector	Example	Selects
Element	<code>p { }</code>	All <code><p></code> tags
Class (dot)	<code>.note { }</code>	Everything with <code>class="note"</code>
ID (hash)	<code>#title { }</code>	The one element with <code>id="title"</code>

Selector	Example	Selects
Group (comma)	<code>h1, h2 { }</code>	Both h1 and h2
Descendant (space)	<code>nav a { }</code>	Links that are inside <code>nav</code>
Universal	<code>* { }</code>	Everything
Hover	<code>a:hover { }</code>	A link while the mouse is over it

TIP: Prefer **classes** for styling. IDs are best kept for JavaScript and page jumps.

3. Colors and Backgrounds

WHAT? Properties that paint text and backgrounds.

WHY? Color creates mood and makes text readable.

HOW? Use color names, HEX codes, or RGB.

```
body {
  background-color: #f4f4f4;    /* HEX */
}
h1 {
  color: navy;                 /* name */
}
p {
  color: rgb(60, 60, 60);      /* RGB */
  background-color: rgba(255, 200, 0, 0.3); /* 0.3 = 30% opaque */
}
.hero {
  background-image: url("bg.jpg");
  background-size: cover;
}
```

4. Text and Fonts

WHAT? Properties that control how text looks.

WHY? Good typography makes reading comfortable.

HOW? Apply them to text elements or the whole `body`.

```
body {
  font-family: Arial, sans-serif; /* backup fonts after the comma */
  font-size: 16px;
  line-height: 1.6;              /* space between lines */
}
h1 {
  font-weight: bold;
  text-align: center;             /* left | right | center | justify */
  text-transform: uppercase;
  letter-spacing: 2px;
  text-decoration: underline;    /* or none: removes link underline */
}
```

5. Units: px, %, em, rem, vw

WHAT? Units tell CSS how big something is.

WHY? Choosing the right unit decides whether your page adapts to different screens.

HOW?

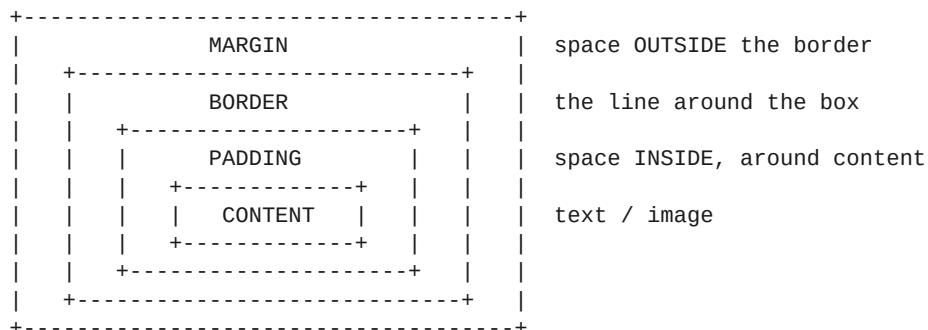
Unit	Meaning	Good for
px	Fixed pixels	Borders, small details
%	Percent of the parent	Widths that adapt
em / rem	Multiple of font size (rem = root size)	Text and spacing
vw / vh	1% of screen width / height	Full-screen sections

6. The Box Model (Most Important!)

WHAT? Every HTML element is a rectangular **box**. The box has four layers: content, padding, border and margin.

WHY? Understanding the box tells you exactly where space comes from and why elements sit where they do.

HOW? Use these properties:



```
.card {
  width: 300px;
  padding: 20px;           /* inside space */
  border: 2px solid #333;  /* thickness style color */
  margin: 15px;           /* outside space */
  border-radius: 10px;     /* rounded corners */
  box-sizing: border-box;  /* width includes padding + border */
}
/* Shortcut order: top right bottom left (clockwise) */
.a { margin: 10px 20px 10px 20px; }
.b { margin: 10px 20px; }      /* top/bottom left/right */
.c { margin: 0 auto; }         /* centers a block horizontally */
```

TIP: Add `* { box-sizing: border-box; }` at the top of every CSS file. It saves many layout headaches.

7. Display: block, inline, inline-block, none

WHAT? The **display** property decides how an element behaves in the flow of the page.

WHY? It controls whether things sit on separate lines or side by side, and whether they appear at all.

HOW?

Value	Behaviour	Examples
block	New line, full width, width/height work	div, p, h1
inline	Sits in the line, width/height ignored	span, a, strong
inline-block	In the line, but width/height work	

Value	Behaviour	Examples
none	Hidden completely	
flex	Flexible layout (next topic)	

8. Flexbox: Easy Layouts

WHAT? Flexbox is a layout system that arranges child items in a row or column inside a parent container.

WHY? Before flexbox, centering and aligning items was painful. Now menus, cards and page layouts are easy.

HOW? Set **display: flex** on the **parent**. The children then line up automatically.

```
<div class="row">
  <div class="box">1</div>
  <div class="box">2</div>
  <div class="box">3</div>
</div>

.row {
  display: flex;
  flex-direction: row;           /* row | column */
  justify-content: space-between; /* start|center|end|space-between */
  align-items: center;           /* cross axis: top-to-bottom for a row */
  gap: 15px;                     /* space between items */
  flex-wrap: wrap;               /* wrap to next line if full */
}
.box { flex: 1; }                /* every box shares space equally */

/* Perfect centering of anything: */
.center {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
```

9. Position

WHAT? The **position** property lets you place an element at an exact spot.

WHY? Badges, sticky menus, pop-ups and overlays need it.

HOW? Set the position type, then use **top**, **right**, **bottom**, **left**.

Value	Meaning
static	Default. Normal flow.
relative	Moves relative to its normal spot; also becomes a reference for children.
absolute	Placed relative to the nearest positioned parent.
fixed	Stays fixed on screen, even while scrolling.
sticky	Scrolls normally, then sticks at the edge.

```
nav {
  position: sticky;
  top: 0;                /* stays at the top while scrolling */
  background: white;
}
```

10. Hover Effects and Transitions

WHAT? `:hover` styles an element when the mouse is on it. `transition` makes the change smooth.

WHY? Feedback tells the user 'this is clickable' and makes the page feel alive.

HOW?

```
.btn {
  background: royalblue;
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 6px;
  transition: 0.3s;          /* animate over 0.3 seconds */
}
.btn:hover {
  background: darkblue;
  transform: scale(1.1);      /* grow 10% */
}
```

11. Responsive Design (Media Queries)

WHAT? Responsive design means the page adjusts to any screen: phone, tablet or laptop.

WHY? More than half of visitors use mobiles.

HOW? Use flexible widths and a **media query** that applies rules only when the screen is a certain size.

```
.row { display: flex; gap: 10px; }

@media (max-width: 600px) {      /* screens 600px wide or smaller */
  .row { flex-direction: column; } /* stack items vertically */
  h1 { font-size: 24px; }
}
```

Day 2 Project: Style Your Profile Page

Create `style.css`, link it, and add class names to your Day 1 HTML (for example `<nav class="menu">`).

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: Arial, sans-serif;
  background: #f4f6f8;
  color: #333;
  line-height: 1.6;
}
header {
  background: #1F6FB5;
  color: white;
  text-align: center;
  padding: 40px 20px;
}
.menu {
  display: flex;
  justify-content: center;
  gap: 25px;
  background: #14213D;
  padding: 12px;
}
.menu a {
  color: white;
  text-decoration: none;
}
.menu a:hover { color: #FCA311; }
```

```
section {
  max-width: 800px;
  margin: 20px auto;
  padding: 20px;
  background: white;
  border-radius: 10px;
}
footer {
  text-align: center;
  padding: 15px;
  color: #666;
}
@media (max-width: 600px) {
  .menu { flex-direction: column; align-items: center; }
}
```

Day 2 Practice Questions

- What are the 3 ways to add CSS? Which is best?
- Explain the box model with a drawing.
- Difference between **padding** and **margin**?
- Center a box horizontally and vertically using flexbox.
- Make a button that turns red on hover.
- What does **@media (max-width: 600px)** do?

JavaScript

Behaviour and interaction

1. What is JavaScript?

WHAT? JavaScript (JS) is a real **programming language** that runs inside the browser. It is not the same as Java.

WHY? HTML and CSS are static. JavaScript makes pages react: validate forms, show pop-ups, update text, build games, load data.

HOW? Write JS in a separate file and link it at the bottom of `<body>`.

```
<body>
  ...your page...
  <script src="script.js"></script>  <!-- last line inside body -->
</body>
```

Try it now: press F12 in Chrome and open the **Console** tab. It is a safe playground where you can type JavaScript and see results.

```
console.log("Hello from JavaScript!"); // prints to the console
alert("Welcome!");                    // pop-up box
// This is a single-line comment
/* This is a
   multi-line comment */
```

2. Variables

WHAT? A variable is a **named box** that stores a value.

WHY? Programs must remember things: a user's name, a score, a total.

HOW? Create with **let** (value can change) or **const** (value never changes).

```
let age = 18;
age = 19; // allowed: let can change

const college = "ABC College";
// college = "XYZ"; // ERROR: const cannot change

let message = "Hi " + college; // join text with +
console.log(message);
```

- Names cannot start with a number or contain spaces. Use camelCase: `studentName`.
- JavaScript is case-sensitive: `age` and `Age` are different.
- Avoid the old `var`. Use `let` and `const`.

3. Data Types

WHAT? The kind of value a variable holds.

WHY? The computer treats numbers, text and true/false differently.

HOW? The main types:

Type	Example	Meaning
Number	25, 3.14	Any number
String	"Hello", 'Hi'	Text in quotes
Boolean	true, false	Yes / no
Undefined	let x;	Declared but no value
Null	null	Intentionally empty
Array	[1, 2, 3]	List of values
Object	{name: "Arun"}	Group of key/value pairs

```
console.log(typeof 25);      // "number"
console.log(typeof "hello"); // "string"
console.log(typeof true);   // "boolean"
```

4. Operators

WHAT? Symbols that perform actions on values.

WHY? Calculations and comparisons are the core of every program.

HOW?

Kind	Operators	Example
Math	+ - * / % **	10 % 3 gives 1 (remainder)
Assignment	= += -=	x += 5 same as x = x + 5
Comparison	== === != > < >= <=	5 === 5 gives true
Logical	&& (and), (or), ! (not)	age > 18 && hasID

TIP: Always use === (strict equal) rather than ==. "5" == 5 is true but "5" === 5 is false, which is what you usually want.

5. Conditions (if / else)

WHAT? Code that makes decisions.

WHY? Programs need to do different things in different situations, such as pass or fail, login or reject.

HOW? Use **if**, **else if**, **else**.

```
let marks = 72;

if (marks >= 90) {
  console.log("Grade A");
} else if (marks >= 60) {
  console.log("Grade B");
} else {
  console.log("Try again");
}
```

6. Loops

WHAT? A loop repeats code.

WHY? Writing the same line 100 times is silly. A loop does it in 3 lines.

HOW? **for** when you know how many times; **while** when you repeat until something changes.

```
for (let i = 1; i <= 5; i++) {
  console.log("Number " + i);
}
//   start    condition    step

let n = 3;
while (n > 0) {
  console.log(n);
  n--;
} // n = n - 1 (without this: infinite loop!)
```

7. Functions

WHAT? A function is a **reusable block of code** with a name.

WHY? Write once, use many times. Keeps code short and organised.

HOW? Define it with **function**, then call it by its name with **()**.

```
function greet(name) {           // name = parameter (input)
  return "Hello, " + name;       // return = output
}
console.log(greet("Arun"));      // Hello, Arun
console.log(greet("Meena"));     // Hello, Meena

function add(a, b) {
  return a + b;
}
let total = add(5, 7);           // total is 12

// Arrow function: a shorter way
const square = (x) => x * x;
console.log(square(4));          // 16
```

8. Arrays

WHAT? An array is an ordered **list** of values in one variable.

WHY? Handle many items (names, marks, tasks) without creating 100 variables.

HOW? Use square brackets. Counting starts at **0**.

```
let fruits = ["Apple", "Banana", "Mango"];
console.log(fruits[0]);          // Apple (first item is index 0)
console.log(fruits.length);     // 3

fruits.push("Grapes");          // add to end
fruits.pop();                   // remove from end

for (let f of fruits) {         // go through each item
  console.log(f);
}
```

9. Objects

WHAT? An object groups related data as **key: value** pairs.

WHY? One student has a name, age and course; an object keeps them together.

HOW? Use curly braces. Access with a dot.

```
let student = {
  name: "Arun",
  age: 18,
  course: "BCA"
};
```

```
console.log(student.name); // Arun
student.age = 19;          // change a value
```

10. The DOM: Changing the Web Page

WHAT? DOM means **Document Object Model**. The browser turns your HTML into a tree of objects that JavaScript can read and change.

WHY? This is where JavaScript and HTML meet. It lets you change text, styles and elements while the page is open.

HOW? First **select** the element, then **change** it.

```
<h1 id="title">Hello</h1>
<p class="note">Some text</p>

let title = document.getElementById("title"); // by id
let note  = document.querySelector(".note");  // by CSS selector

title.textContent = "Welcome!"; // change text
title.style.color = "red";      // change style
note.classList.add("highlight"); // add a CSS class
```

11. Events

WHAT? An event is something that happens on the page: click, key press, mouse move, form submit.

WHY? Interaction is the whole point. We run code when the user does something.

HOW? Use `addEventListener(eventName, function)` on an element.

```
let btn = document.querySelector("#myBtn");

btn.addEventListener("click", function () {
  alert("Button was clicked!");
});
```

Event	Fires when
click	User clicks
dblclick	Double-click
mouseover	Mouse enters element
keydown	A key is pressed
input	User types in a field
submit	A form is submitted

Day 3 Mini Projects

Project A: Click Counter

```
<!-- HTML -->
<h2>Count: <span id="count">0</span></h2>
<button id="plus">+</button>
<button id="minus">-</button>
<script src="script.js"></script>

// script.js
let count = 0;
const show = document.getElementById("count");

document.getElementById("plus").addEventListener("click", () => {
```

```

    count++;
    show.textContent = count;
  });
  document.getElementById("minus").addEventListener("click", () => {
    count--;
    show.textContent = count;
  });

```

Project B: Dark Mode Button

```

/* CSS */
body.dark { background: #111; color: white; }

<!-- HTML -->
<button id="mode">Toggle Dark Mode</button>

// JS
document.getElementById("mode").addEventListener("click", () => {
  document.body.classList.toggle("dark"); // add if missing, remove if present
});

```

Project C: To-Do List

```

<!-- HTML -->
<input type="text" id="task" placeholder="New task">
<button id="add">Add</button>
<ul id="list"></ul>

// JS
const input = document.getElementById("task");
const list = document.getElementById("list");

document.getElementById("add").addEventListener("click", () => {
  if (input.value === "") {
    alert("Please type a task");
    return;
  }
  const li = document.createElement("li"); // make a new <li>
  li.textContent = input.value; // put the text inside
  li.addEventListener("click", () => li.remove()); // click to delete
  list.appendChild(li); // add it to the page
  input.value = ""; // clear the box
});

```

Project D: Simple Form Validation

```

<form id="f">
  <input type="text" id="user" placeholder="Username">
  <p id="msg" style="color:red"></p>
  <button>Login</button>
</form>

// JS
document.getElementById("f").addEventListener("submit", (e) => {
  e.preventDefault(); // stop the page reloading
  const name = document.getElementById("user").value;
  if (name.length < 4) {
    document.getElementById("msg").textContent = "At least 4 characters";
  } else {
    document.getElementById("msg").textContent = "";
    alert("Welcome " + name);
  }
});

```

Day 3 Practice Questions

- Difference between **let** and **const**?

- Write a loop that prints 1 to 10.
- Write a function that returns the bigger of two numbers.
- What is the DOM?
- How do you run code when a button is clicked?
- Add a button to your profile page that changes its background color.

Common Beginner Mistakes

Problem	Reason and fix
Nothing changes in the browser	You forgot to save the file, or forgot to refresh (F5).
CSS not working	Check the <code><link></code> path and file name. Check each line ends with <code>;</code> and each block has <code>}</code> .
Image not showing	Wrong file name or extension (<code>photo.jpg</code> vs <code>Photo.JPG</code>). Keep it in the same folder.
JS: "is null" error	Script tag is in <code><head></code> . Move it to the end of <code><body></code> , or check the id spelling.
Tags not closed	Every <code><div></code> needs a <code></div></code> . Indent your code so you can see pairs.
Quotes mismatch	Open with <code>"</code> and close with <code>"</code> . Do not mix straight and curly quotes.
Spelling and case	<code>getElementById</code> is not <code>getelementbyid</code> . JS is case-sensitive.

How to debug (find mistakes)

- Press **F12** in Chrome. Red errors in **Console** tell you the file name and line number.
- Use `console.log()` to print values and see what your code is doing.
- Right-click any element > **Inspect** to see its HTML and CSS live.
- Change one thing at a time and test after each change.

One-Page Cheat Sheet

HTML

Need	Code
Heading / paragraph	<code><h1> ... <h6>, <p></code>
Link	<code>text</code>
Image	<code></code>
List	<code>.. / </code>
Table	<code><table><tr><th>/<td></code>
Form	<code><form>, <input>, <label>, <button></code>
Layout	<code><header> <nav> <main> <section> <footer> <div></code>

CSS

Need	Code
Select	<code>p .class #id a:hover</code>
Color	<code>color, background-color</code>
Text	<code>font-size, font-family, text-align</code>
Box	<code>width, padding, border, margin</code>
Layout	<code>display:flex; justify-content; align-items; gap</code>

Need	Code
Mobile	<code>@media (max-width: 600px) { }</code>

JavaScript

Need	Code
Variable	<code>let x = 5; const y = 10;</code>
If	<code>if (x > 3) { } else { }</code>
Loop	<code>for (let i=0; i<5; i++) { }</code>
Function	<code>function name(a) { return a; }</code>
Select element	<code>document.querySelector("#id")</code>
Change text	<code>el.textContent = "new"</code>
Event	<code>el.addEventListener("click", () => { })</code>

Glossary of Simple Words

Word	Meaning
Browser	Program that opens websites (Chrome, Edge, Firefox)
Tag	A keyword in angle brackets, like <code><p></code>
Attribute	Extra info inside a tag, like <code>src</code>
Selector	CSS part that chooses which element to style
Property	What you change in CSS, like <code>color</code>
Variable	A named box for storing a value
Function	A named, reusable block of code
DOM	Browser's live tree of your HTML
Event	Something that happens: click, key press
Responsive	A page that fits every screen size

What to Learn Next

- Practice: rebuild simple pages you like (a menu, a card, a login page).
- CSS Grid, animations and Bootstrap or Tailwind.
- JavaScript: fetching data from APIs, `localStorage`, ES6 features.
- Git and GitHub to save and share your projects, then publish free on GitHub Pages.
- A framework such as React, once JavaScript basics feel comfortable.

REMEMBER: Coding is a skill like cycling or swimming. You learn by doing, not by reading. Type every example yourself, break it on purpose, then fix it. Happy coding!

Git

GitHub

Netlify

Save, Share and Publish Your First Website

Git + GitHub + Netlify for Absolute Beginners

Companion to the 3-Day Web Development Handbook | What? Why? How?

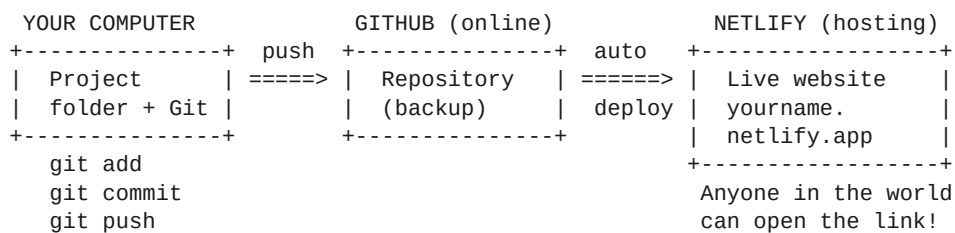
No prior knowledge needed. Every command is explained from zero.

Welcome!

In the 3-day course you built a web page. But it lives only on **your computer**. This handbook teaches you three tools that take it to the whole world:

Tool	One-line meaning	Problem it solves
Git	A tool on your computer that saves versions of your code	'I broke my code and can't go back!'
GitHub	A website that stores your Git projects online	'My laptop crashed, my work is gone!' / 'How do I share my code?'
Netlify	A service that hosts your site on the internet	'How does my friend open my website?'

The Big Picture



Real-Life Analogy

Real life	Tool
Writing a diary and putting a bookmark at every finished chapter	Git (commits are bookmarks)
Keeping a copy of the diary in a locker at the bank	GitHub (safe online copy)
Printing the diary as a book and placing it in a public library	Netlify (everyone can read it)

Before You Start: Setup (15 minutes)

- Finish your profile page from the 3-day course. Keep all files in one folder, for example **web-course**.
- **Install Git**: download from git-scm.com. On Windows keep all default options; it also installs **Git Bash**, a terminal you will use.
- Check it worked. Open a terminal (Git Bash on Windows, Terminal on Mac/Linux) and type the command below.
- **Create a GitHub account** at github.com (free). Choose a professional username, because it appears in your links.
- **Create a Netlify account** at netlify.com. Choose 'Sign up with GitHub' so the two are already connected.

```
git --version
# Expected output (your number may differ):
# git version 2.45.1
```

TIP: If `git --version` says 'command not found', close the terminal, reopen it and try again. If it still fails, reinstall Git.

PART 1

Git

Save versions of your code on your computer

1. What is Git?

WHAT? Git is a **version control system**. It records snapshots of your project each time you tell it to. Each snapshot is called a **commit**.

WHY? Without Git, students end up with folders like `project_final`, `project_final2`, `project_REAL_final`. With Git there is one folder and a full history. You can go back in time, see what changed and who changed it, and try new ideas safely.

HOW? You use short commands in the terminal. Git watches a folder called a **repository** (repo).

- **Repository (repo)** = a project folder that Git is tracking.
- **Commit** = a saved snapshot with a message, like a save point in a video game.
- **Branch** = a separate line of work, so experiments don't disturb the main code.

2. Terminal Basics (needed for Git)

WHAT? The terminal is a window where you control the computer by typing commands instead of clicking.

WHY? Git is used through commands. A few navigation commands are all you need.

HOW? Open the terminal, then use the commands below. Lines starting with `#` are comments, so do not type them.

Command	Meaning	Example
<code>pwd</code>	Print where I am now	<code>pwd</code>
<code>ls</code>	List files in this folder	<code>ls</code>
<code>cd folder</code>	Go into a folder	<code>cd Desktop/web-course</code>
<code>cd ..</code>	Go back one folder	<code>cd ..</code>
<code>mkdir name</code>	Make a new folder	<code>mkdir my-site</code>
<code>clear</code>	Clean the screen	<code>clear</code>

```
cd Desktop/web-course    # go to your project folder
ls                        # you should see index.html, style.css ...
```

TIP: In VS Code press `Ctrl + `` (backtick) to open a terminal that already sits inside your project folder. This is the easiest way.

3. One-Time Setup: Tell Git Who You Are

WHAT? Git stamps every commit with your name and email.

WHY? So everyone can see who made each change.

HOW? Run these once per computer. Use the same email as your GitHub account.

```
git config --global user.name "Arun Kumar"
git config --global user.email "arun@example.com"
git config --global init.defaultBranch main

git config --list          # check your settings
```

4. The Three Areas of Git (Very Important)

WHAT? Git moves your files through three areas.

WHY? Understanding this makes every command make sense. You choose exactly what goes into each snapshot.

HOW? Think of taking a family photo:

WORKING FOLDER (you edit files)		STAGING AREA (get ready)		REPOSITORY (history) (saved snapshots)
+-----+ index.html style.css +-----+	add	+-----+ index.html +-----+	commit	+-----+ Commit 1 Commit 2 ... +-----+
people wandering around		people standing in position		the photo is taken

5. Start Tracking: git init

WHAT? `git init` turns an ordinary folder into a Git repository.

WHY? Git ignores folders until you say 'watch this one'.

HOW? Go inside your project folder and run it once.

```
cd Desktop/web-course
git init
# Initialized empty Git repository in .../web-course/.git/
```

This creates a hidden folder named `.git` where Git stores all history. **Never delete it**, and never run `git init` inside another repo.

IMPORTANT: Important for hosting later: rename `profile.html` to `index.html`. Browsers and hosts open a file named `index.html` first, as the home page.

6. Check the Situation: git status

WHAT? It shows which files are new, changed, staged or unchanged.

WHY? You always need to know the current state before acting. Use it constantly!

HOW? Just type it. Red = not staged yet, green = staged.

```
git status
# On branch main
# No commits yet
# Untracked files:
#   index.html
#   style.css
#   script.js
```

7. Stage Files: git add

WHAT? `git add` moves files from your working folder into the staging area.

WHY? You pick exactly which changes go into the next snapshot.

HOW? Name a file, or use a dot to add everything.

```
git add index.html      # add one file
git add style.css script.js  # add several
git add .               # add everything in this folder

git status              # now the files show in green
```

8. Save a Snapshot: git commit

WHAT? `git commit` permanently saves everything in the staging area with a message.

WHY? This is your save point. If you break something tomorrow, you can return to today.

HOW? Use `-m` followed by a short message in quotes.

```
git commit -m "Add profile page with HTML and CSS"
# [main (root-commit) a1b2c3d] Add profile page with HTML and CSS
# 3 files changed, 120 insertions(+)
```

Writing good commit messages

Bad	Good
"changes", "asdf", "final"	"Add navigation menu"
"fixed stuff"	"Fix broken image path in About section"
"update"	"Style contact form with flexbox"

Rule: start with a verb (Add, Fix, Update, Remove) and describe **what** you did in under 60 characters.

9. See History: git log and git diff

WHAT? `git log` lists all commits. `git diff` shows exactly which lines you changed but haven't staged yet.

WHY? To review your work and find what changed before saving.

HOW?

```
git log                # full history (press q to exit)
git log --oneline      # compact, one line per commit
# a1b2c3d Add profile page with HTML and CSS

git diff              # + green lines added, - red lines removed
```

10. Ignore Files: .gitignore

WHAT? A text file named `.gitignore` lists files and folders Git should never track.

WHY? Some files should not be saved or shared: system junk, secret keys, huge auto-generated folders.

HOW? Create a file named `.gitignore` in the project root and list one pattern per line.

```
# .gitignore
.DS_Store
Thumbs.db
node_modules/
.env
```

11. Undo Mistakes Safely

WHAT? Commands that take back changes.

WHY? Everyone makes mistakes. Git is a safety net.

HOW? Pick the command that matches your situation:

Situation	Command
I edited a file and want to throw the change away	<code>git restore index.html</code>
I staged a file by mistake	<code>git restore --staged index.html</code>
I want to fix the message of my last commit	<code>git commit --amend -m "New message"</code>
I want to undo an old commit safely (keeps history)	<code>git revert a1b2c3d</code>

CAREFUL: `git restore index.html` permanently discards your unsaved edits in that file. Be sure before you run it.

12. Branches

WHAT? A branch is a separate copy of the timeline where you can experiment.

WHY? You can build a new feature without risking the working version. If it works, merge it back; if not, delete it.

HOW? The default branch is called `main`.

```
git branch          # list branches (* marks current)
git switch -c dark-mode  # create and move to a new branch
# ...edit files, git add, git commit...
git switch main      # go back to main
git merge dark-mode  # bring the new work into main
git branch -d dark-mode  # delete the finished branch
```

```
main:      A---B-----F      <- F = merge commit
           \         /
dark-mode:  C---D---E
```

The Everyday Git Cycle

1. Edit your files in VS Code
2. `git status` (what changed?)
3. `git add .` (choose everything)
4. `git commit -m "message"` (save the snapshot)
... repeat ...

Part 1 Practice

- Initialize a repo in your project and make your first commit.
- Change the page title, then use `git diff` to see the change, then commit it.
- Create a branch `experiment`, change the background color, commit, switch back to `main` and see the color return. Then merge.
- Run `git log --oneline`. How many commits do you have?

PART 2

GitHub

Back up and share your code online

1. What is GitHub?

WHAT? GitHub is a website that stores Git repositories in the cloud. **Git is the tool; GitHub is the website.**

WHY? It gives you a backup, lets you work from any computer, shows your work to teachers and employers (it's your coding portfolio), and lets teams collaborate. It is also how Netlify will get your code.

HOW? Push your local commits up to a GitHub repository. Pull others' changes down.

Git	GitHub
Software installed on your computer	Website (github.com)
Works offline	Needs the internet
Tracks versions	Stores versions online and adds sharing tools

Similar websites exist (GitLab, Bitbucket), but GitHub is the most popular.

2. Create a Repository on GitHub

WHAT? A remote repository is an empty online folder waiting for your code.

WHY? It is the destination for your local commits.

HOW? Follow these steps:

- Log in to github.com. Click the + icon (top right) then **New repository**.
- Repository name: **my-profile-site** (lowercase, no spaces, use hyphens).
- Choose **Public** so anyone can see it (best for a portfolio).
- Do **not** tick 'Add a README', 'gitignore' or 'license' now, because you already have local files. An empty repo avoids conflicts.
- Click **Create repository**. GitHub shows a page with the repository address (URL). Copy it.

`https://github.com/YOUR-USERNAME/my-profile-site.git`

3. Logging In from Your Computer

WHAT? Proving to GitHub that you own the account when you push from the terminal.

WHY? Otherwise anyone could change your projects.

HOW? GitHub no longer accepts your account password in the terminal. Use one of these:

Option	How
Browser login (easiest, Windows)	Git for Windows includes Git Credential Manager. On your first push a window opens; choose 'Sign in with your browser' and approve.

Option	How
GitHub CLI	Install <code>gh</code> from cli.github.com , run <code>gh auth login</code> , and follow the prompts.
Personal access token	GitHub: Settings > Developer settings > Personal access tokens. Create one with the <code>repo</code> permission and paste it when asked for a password. Save it safely; GitHub shows it once.

SECURITY: Never share your token or upload it to a repository. Treat it like a password.

4. Connect and Push Your Project

WHAT? **Remote** = the online address linked to your local repo. **Push** = upload your commits to it.

WHY? This copies your work to the cloud.

HOW? Run these commands inside your project folder (after your first commit):

```
git remote add origin https://github.com/YOUR-USERNAME/my-profile-site.git
git branch -M main
git push -u origin main
```

Part	Meaning
<code>remote add origin URL</code>	Give the online address the nickname <code>origin</code>
<code>branch -M main</code>	Make sure the branch is named <code>main</code>
<code>push</code>	Upload commits
<code>-u origin main</code>	Remember this link, so next time <code>git push</code> alone works

Refresh your GitHub page. Your files are now online!

5. The Daily Workflow

WHAT? The routine you will repeat every time you change your project.

WHY? Small, frequent commits keep your work safe and history clean.

HOW?

```
# 1. Edit your files
# 2. Then run:
git add .
git commit -m "Describe what you changed"
git push
```

6. Pull and Clone

WHAT? `git pull` downloads new commits from GitHub. `git clone` downloads a whole repository for the first time.

WHY? Use pull when you work on two computers or in a team. Use clone to get a copy of any public project or to continue on a new laptop.

HOW?

```
git clone https://github.com/YOUR-USERNAME/my-profile-site.git
cd my-profile-site
```

```
git pull # get the latest changes from GitHub
```

```
Computer A --push--> GitHub <--pull-- Computer B
```

7. README: The Front Page of Your Project

WHAT? `README.md` is a text file GitHub shows on your repo's home page. `.md` means **Markdown**, a simple way to format text.

WHY? It tells visitors what your project is and how to use it. A good README impresses teachers and recruiters.

HOW? Create `README.md` in the project folder, commit and push.

```
# My Profile Site
```

```
A personal profile page built with HTML, CSS and JavaScript.
```

```
## Features
```

- Responsive layout
- Dark mode button
- Contact form

```
## Live Demo
```

```
https://YOUR-SITE.netlify.app
```

```
## Author
```

```
Arun Kumar
```

Markdown	Result
<code># Title</code>	Big heading (<code>##</code> smaller)
<code>bold</code>	bold text
<code>- item</code>	bullet list
<code>[text](url)</code>	link
<code>![alt](img.png)</code>	image

8. Collaboration Basics

WHAT? Ways many people work on one project.

WHY? Real jobs are done in teams. These are the tools teams use.

HOW?

Word	Meaning
Fork	Your own copy of someone else's repo on GitHub
Pull Request (PR)	A request: 'please add my changes to your project'. Others review before merging.
Issue	A task or bug report on a repo
Merge conflict	Two people edited the same lines. Git asks you to choose.
Contributor	Someone who adds to a project

Fixing a merge conflict

```
<<<<<<< HEAD
<h1>Welcome to my site</h1>
=====
<h1>Hello, visitor!</h1>
>>>>>>> dark-mode
```

Git marks both versions. Delete the markers (<<<<<<, =====, >>>>>>), keep the line you want, then run `git add .` and `git commit`.

9. Common Errors and Fixes

Message	Cause and fix
<code>fatal: not a git repository</code>	You are in the wrong folder. Use <code>cd</code> to enter the project, or run <code>git init</code> .
<code>Author identity unknown</code>	Run the two <code>git config --global user.name / user.email</code> commands.
<code>remote origin already exists</code>	Use <code>git remote set-url origin NEWURL</code> to change it.
<code>rejected ... fetch first</code>	GitHub has commits you lack. Run <code>git pull</code> , then <code>git push</code> .
<code>nothing to commit</code>	You forgot <code>git add</code> , or nothing changed.
<code>Authentication failed</code>	Use browser login or a personal access token, not your password.
Pressed a key and stuck in a strange screen	Press <code>q</code> to quit, or <code>Esc</code> then <code>:q</code> then Enter.

Part 2 Practice

- Create a repo on GitHub and push your profile site.
- Add a README with a title, description and feature list. Push it.
- Edit a file on the GitHub website, then run `git pull` on your computer.
- Ask a friend to fork your repo and send you a pull request.

Netlify

Host your website on the internet

1. What are Hosting and Netlify?

WHAT? **Hosting** means storing your website files on a computer (server) that is always on and connected to the internet. **Deploying** means putting your files there. **Netlify** is a company that gives free hosting for static websites (HTML, CSS, JS).

WHY? Your laptop is not a server. If you send your friend `index.html` on WhatsApp they only see a file. Hosting gives your site a real link that works on any phone or laptop. Netlify's free plan is generous, gives free HTTPS (the padlock) and needs no server setup.

HOW? Give Netlify your project folder (by dragging it in, or by linking your GitHub repo). It publishes it at an address like `https://yourname.netlify.app`.

Term	Meaning
Static site	Website made only of HTML, CSS, JS and images. No database.
Deploy	Publish a version of your site
Domain / URL	Address of the site (<code>yourname.netlify.app</code>)
HTTPS	Secure connection (padlock). Netlify adds it for free.
Continuous deployment	Every <code>git push</code> automatically updates the live site

2. Pre-Deployment Checklist

WHAT? Things to fix before you publish.

WHY? Most deployment failures are caused by small file mistakes.

HOW? Tick every point:

- The home page file is named exactly `index.html` and sits at the **top level** of the project folder.
- Use **relative paths**: `` and NOT `C:\Users\Arun\Desktop\me.jpg`.
- File names are **lowercase, no spaces**: `my-photo.jpg` not `My Photo.JPG`. Netlify's servers are case-sensitive, so `Photo.jpg` and `photo.jpg` are different files.
- Your links match the real file names exactly (`style.css`, `script.js`).
- Everything works when you open `index.html` on your own computer first.

```
my-profile-site/
|-- index.html      <- must be here, with exactly this name
|-- style.css
|-- script.js
|-- images/
|   |-- me.jpg
|-- README.md
```

3. Method A: Drag and Drop (Fastest, 2 Minutes)

WHAT? Uploading your folder directly to Netlify without using Git.

WHY? Perfect for your very first site. No commands needed.

HOW? Steps:

- Log in at app.netlify.com.
- Go to your **Projects** (or **Sites**) page. Click **Add new project** (older screens say 'Add new site') and choose **Deploy manually**. You can also open app.netlify.com/drop directly.
- Drag your **whole project folder** (the one containing `index.html`) into the dashed drop area. A Chrome-based browser works best.
- Wait a few seconds. Netlify shows a random address such as `https://sparkly-unicorn-123abc.netlify.app`.
- Click the link. Your site is live! Send it to your friends.

TIP: To update a drag-and-drop site, open the site's **Deploys** page and drop the updated folder again. This method is not connected to GitHub, so Method B is better for long-term projects.

4. Method B: Deploy from GitHub (Recommended)

WHAT? Linking your GitHub repository to Netlify. This is called **continuous deployment**.

WHY? After the one-time setup you do nothing extra. Each `git push` republishes the site automatically. This is how professional teams work.

HOW? Steps:

- Push your project to GitHub first (Part 2). Make sure `index.html` is in the repo.
- In Netlify click **Add new project** (or 'Add new site') then **Import an existing project**.
- Choose **GitHub**. Sign in and click **Authorize Netlify**. If your repo is missing, click 'Configure the Netlify app on GitHub' and give access to that repo.
- Select your repository, for example `my-profile-site`.
- Deploy settings: **Branch to deploy** = `main`. **Build command** = leave empty. **Publish directory** = leave empty or type `.` (a dot means the root folder). Static sites need no build.
- Click **Deploy**. In about 20 seconds the deploy shows `*Published*`.
- Click the site URL at the top of the project page to open your live site.

Setting	Value for a plain HTML/CSS/JS site
Branch to deploy	<code>main</code>
Build command	(empty)
Publish directory	(empty) or <code>.</code>

TIP: Netlify sometimes renames buttons and menus. If a label looks different, look for the option that means 'add a project from GitHub'. The idea stays the same.

5. Update Your Live Site

WHAT? Changing your site after it is live.

WHY? Websites keep evolving. With Method B updating is just a push.

HOW? Edit, commit and push as usual:

```
# edit index.html in VS Code, then:
git add .
git commit -m "Add new project section"
git push
# Netlify detects the push and republishes automatically.
```

Open the **Deploys** tab in Netlify to watch progress. If you don't see the change, hard refresh the browser with **Ctrl + F5** (Cmd + Shift + R on Mac).

Rollback: go back to a working version

In **Deploys**, open an older successful deploy and click **Publish deploy**. Your site instantly returns to that version.

6. Change the Site Name

WHAT? The random name Netlify gives (like `sparkly-unicorn-123abc`) can be replaced.

WHY? A clean link looks professional on your resume.

HOW? Steps:

- Open your site in Netlify and go to **Site configuration** (or **Site settings**).
- Under **Site details** find **Change site name**.
- Type a name such as `arun-profile`. If it is free, the link becomes `https://arun-profile.netlify.app`.

7. Custom Domain (Optional)

WHAT? Using your own address like `arunkumar.com` instead of `.netlify.app`.

WHY? Looks more professional and easier to remember.

HOW? Buy a domain from a registrar (for example Namecheap, GoDaddy), then in Netlify open **Domain management** and click **Add a domain**. Netlify shows the DNS settings to copy into your registrar. Changes can take up to 48 hours. HTTPS is added automatically.

TIP: Domains cost money each year. The free `.netlify.app` link is enough for a college portfolio.

8. Troubleshooting Deployments

Problem	Reason and fix
Page not found (404) on the home page	<code>index.html</code> is missing, misnamed (<code>Index.html</code> , <code>index.htm</code>) or inside a sub-folder. Move it to the root and redeploy.
Site opens, but no styling	CSS path is wrong or case differs. Check <code><link href="style.css"></code> and the real file name.
Images missing	Absolute path (<code>C:\...</code>) or wrong upper/lower case. Use relative paths like <code>images/me.jpg</code> .
JavaScript not working	Check the <code><script src></code> path. Press F12 and read the Console errors.
Changes not showing after push	Check Deploys is 'Published', then hard refresh (Ctrl + F5). Make sure you pushed to <code>main</code> .
Deploy shows Failed	Open the deploy log. For a plain site, make sure Build command is empty.
Repo not listed while importing	In Netlify choose 'Configure the Netlify app on GitHub' and grant repo access.
Drag and drop stuck on 'Uploading'	Site too big, or browser problem. Keep it small, try Chrome, or use Method B.

Your Final Project Checklist

- ☐ Profile site finished and working locally, with `index.html` at the root.
- ☐ `git init`, first commit made.
- ☐ Repo created on GitHub and code pushed.
- ☐ README.md added with description and live link.
- ☐ Site deployed on Netlify from GitHub.
- ☐ Site name changed to something clean.
- ☐ Made one edit, pushed, and saw the live site update by itself.
- ☐ Shared the link with a friend and asked them to open it on a phone.

Git and GitHub Cheat Sheet

First-time setup

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

Start a project and push to GitHub

```
git init
git add .
git commit -m "First commit"
git remote add origin https://github.com/USER/REPO.git
git branch -M main
git push -u origin main
```

Every day

```
git status
git add .
git commit -m "What I changed"
git push
git pull
```

Command	What it does
<code>git status</code>	Show state of files
<code>git add .</code>	Stage all changes
<code>git commit -m "msg"</code>	Save a snapshot
<code>git log --oneline</code>	Short history
<code>git diff</code>	Show unsaved changes
<code>git push / git pull</code>	Upload / download commits
<code>git clone URL</code>	Copy a repo to your computer
<code>git switch -c name</code>	New branch
<code>git merge name</code>	Merge branch into current
<code>git restore file</code>	Discard edits in a file
<code>git revert id</code>	Undo a commit safely

Glossary of Simple Words

Word	Meaning
Repository	A project folder tracked by Git
Commit	A saved snapshot with a message
Staging area	Waiting room for changes before a commit
Branch	Separate line of work
Remote / origin	The online copy of the repo (on GitHub)
Push / Pull	Upload / download commits

Word	Meaning
Clone	Copy a repo for the first time
Fork	Your own copy of someone else's repo
Pull request	Request to merge your changes into a project
Deploy	Publish your site
Hosting	Server that keeps your site online
Static site	Site with only HTML, CSS, JS and images

What to Learn Next

- Use GitHub Pages as a second free way to host (repo Settings > Pages).
- Learn Git branching with pull requests by working on a small project with a friend.
- Use the Netlify CLI and Netlify Forms to make your contact form really send messages.
- Add a custom domain and a portfolio with several projects.
- Explore frameworks (React) and build tools; Netlify supports them with a build command.

REMEMBER: Commit early, commit often, and push at the end of every study session. Your future self will thank you. Happy coding!

NANDHA ENGINEERING COLLEGE

ERODE

Web Development | HTML, CSS, JavaScript + Git, GitHub and Netlify

ACADEMIC YEAR 2026 – 2027

MANUAL

Prepared By

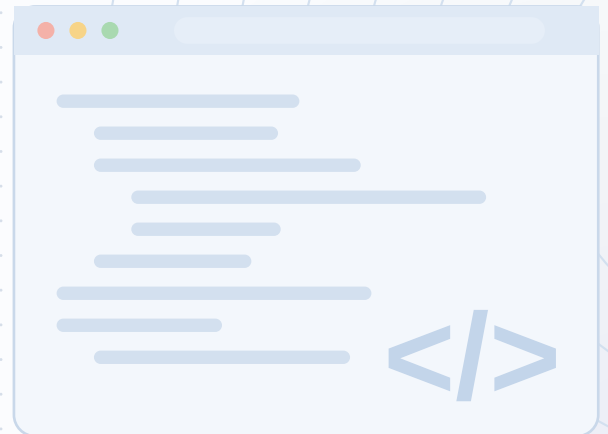
1



A. Satheesh Kumar

Assistant Professor

Department of CSE
Nandha Engineering College, Erode



2



M. Joseph Vinson Samuel

Teaching Assistant

Department of CSE
Nandha Engineering College, Erode

`git commit -m "deploy"`